StockFlow – Backend Engineering Intern Case Study

Candidate: Shubham Kadam

Introduction & Assumptions

This document addresses all three parts of the ***StockFlow*** case study. Where requirements were incomplete, I've made explicit, reasoned assumptions rather than leaving gaps — the same way I'd handle ambiguity in a real engineering team.

Global Assumptions:

- **Database:** PostgreSQL (ACID-compliant, production standard for financial/inventory data)
- **Framework:** Python + Flask with SQLAlchemy ORM
- Authentication exists (JWT-based) but is out of scope for this exercise
- All timestamps are stored in UTC

Part 1: Code Review & Debugging

1.1 Issues Identified

A. No Input Validation

The code directly accesses data['name'], data['sku'], etc. without checking if the fields exist or are the right type. If any field is missing, Python throws a KeyError, the app crashes with a 500 error instead of returning a helpful 400.

B. SKU Uniqueness Not Enforced at Application Level

The code doesn't check if the SKU already exists before inserting. Even if there's a DB-level unique constraint, the error will be a raw IntegrityError that bubbles up unhandled. Without the constraint, duplicate SKUs silently corrupt inventory tracking, reporting, and order management.

C. Two Separate Transactions (Risk of Partial Writes)

There are two db.session.commit() calls. If the server crashes, loses DB connection, or throws an error between the first and second commit, you get a Product row with no corresponding Inventory row. The product exists in the system but is invisible in stock queries , silent data corruption.

D. warehouse_id Stored on Product (Wrong Data Model)

The Product model stores a single warehouse_id, but the requirements state products can exist in multiple warehouses. This makes multi-warehouse support structurally impossible without a schema change later.

E. Price Stored Without Decimal Safety

data['price'] is passed directly. If JSON sends 9.99, Python may store it as a float (9.989999999...), which causes rounding errors in financial calculations. Price must use Decimal.

F. No Error Handling or Logging

Any exception crashes the endpoint with a raw 500. No logging means production issues are invisible. No rollback means the DB session is left in a dirty state.

G. No HTTP Status Codes

Success returns 200 OK instead of 201 Created. Errors return no meaningful codes. This breaks API contracts with frontend/mobile clients.

H. No Idempotency

Retrying a failed request (e.g., network timeout) creates duplicate products. There's no safeguard against double-submission.

1.2 Impact Analysis

Issue	Production Impact
No validation	500 crashes on bad input; corrupt data enters the DB
Duplicate SKU	Inventory tracking, order routing, and analytics all break silently
Two transactions	Product created with no stock record; stockout alerts fire incorrectly
Wrong data model	Multi-warehouse feature requires emergency schema migration

Float price	Financial reports show rounding errors; billing disputes
No error handling	Exceptions leak stack traces; DB left in dirty state
Wrong status codes	Client apps misinterpret success/failure

1.3 Fixed Implementation

```
import logging
from flask import request, jsonify
from sqlalchemy.exc import IntegrityError
from decimal import Decimal, InvalidOperation

logger = logging.getLogger(__name__)

@app.route('/api/products', methods=['POST'])
def create_product():
    data = request.get_json(silent=True)

    # --- Input Validation ---
    if not data:
        return jsonify({"error": "Request body must be valid JSON"}), 400

    required_fields = ['name', 'sku', 'price', 'warehouse_id', 'initial_quantity']
    missing = [f for f in required_fields if f not in data]
    if missing:
        return jsonify({"error": f"Missing required fields: {', '.join(missing)}"}),
400

    # Type validation
    try:
        price = Decimal(str(data['price']))
        if price < 0:
            raise ValueError("Price cannot be negative")
    except (InvalidOperation, ValueError) as e:
        return jsonify({"error": f"Invalid price: {e}"}), 400

    try:
```

```
        quantity = int(data['initial_quantity'])
        if quantity < 0:
            raise ValueError("Quantity cannot be negative")
    except (ValueError, TypeError):
        return jsonify({"error": "initial_quantity must be a non-negative
integer"}), 400

    # Optional fields with defaults
    description = data.get('description', '')
    product_type = data.get('product_type', 'standard')

    try:
        # Single transaction — both writes succeed or both roll back
        with db.session.begin():

            # Check SKU uniqueness explicitly (better error message than
IntegrityError)
            if Product.query.filter_by(sku=data['sku']).first():
                return jsonify({"error": f"SKU '{data['sku']}' already exists"}),
409

            # Validate warehouse exists and belongs to a company
            warehouse = Warehouse.query.get(data['warehouse_id'])
            if not warehouse:
                return jsonify({"error": f"Warehouse {data['warehouse_id']} not
found"}), 404

            # Create product — no warehouse_id here (product is warehouse-agnostic)
            product = Product(
                name=data['name'],
                sku=data['sku'],
                price=price,
                description=description,
                product_type=product_type
            )
            db.session.add(product)
            db.session.flush()  # Get product.id without committing

            # Create inventory record for this warehouse
            inventory = Inventory(
                product_id=product.id,
                warehouse_id=data['warehouse_id'],
                quantity=quantity
            )
            db.session.add(inventory)
```

```
        # Audit log
        history = InventoryHistory(
            product_id=product.id,
            warehouse_id=data['warehouse_id'],
            change=quantity,
            reason='initial_stock'
        )
        db.session.add(history)

    logger.info(f"Product created: id={product.id}, sku={data['sku']}")

    return jsonify({
        "message": "Product created successfully",
        "product_id": product.id,
        "sku": product.sku
    }), 201  # 201 Created — correct HTTP semantics

except IntegrityError as e:
    logger.error(f"DB integrity error creating product: {e}")
    return jsonify({"error": "Database constraint violation"}), 409

except Exception as e:
    logger.error(f"Unexpected error creating product: {e}", exc_info=True)
    return jsonify({"error": "Internal server error"}), 500
```

Key fixes summary:

- Single db.session.begin() block — atomic, all-or-nothing
- db.session.flush() gets the product.id before committing without a second transaction
- Explicit SKU check returns a 409 Conflict with a useful message
- Decimal(str(price)) avoids float rounding bugs
- warehouse_id removed from Product model — products are warehouse-agnostic
- Audit entry written in the same transaction
- Proper 201 Created response code

Part 2: Database Design

2.1 Schema (PostgreSQL DDL)

```
-- core company entities

CREATE TABLE companies (
```

```sql
   id          SERIAL PRIMARY KEY,

   name        VARCHAR(255) NOT NULL,

   created_at  TIMESTAMP NOT NULL DEFAULT NOW(),

   updated_at  TIMESTAMP NOT NULL DEFAULT NOW()

);

-- warehouses belong to companies

CREATE TABLE warehouses (

   id          SERIAL PRIMARY KEY,

   company_id  INTEGER NOT NULL REFERENCES companies(id) ON DELETE RESTRICT,

   name        VARCHAR(255) NOT NULL,

   location    VARCHAR(500),

   is_active   BOOLEAN NOT NULL DEFAULT TRUE,

   created_at  TIMESTAMP NOT NULL DEFAULT NOW(),

   updated_at  TIMESTAMP NOT NULL DEFAULT NOW()

);

-- products are warehouse agnostic

-- placement tracked in inventory table

CREATE TABLE products (

   id              SERIAL PRIMARY KEY,

   sku             VARCHAR(100) NOT NULL UNIQUE,

   name            VARCHAR(255) NOT NULL,

   description     TEXT,

   price           NUMERIC(12, 4) NOT NULL CHECK (price >= 0),

   product_type    VARCHAR(50) NOT NULL DEFAULT 'standard',

   low_stock_threshold INTEGER NOT NULL DEFAULT 10 CHECK (low_stock_threshold >=
0),
```

```sql
   is_bundle       BOOLEAN NOT NULL DEFAULT FALSE,

   is_active       BOOLEAN NOT NULL DEFAULT TRUE,

   created_at      TIMESTAMP NOT NULL DEFAULT NOW(),

   updated_at      TIMESTAMP NOT NULL DEFAULT NOW()

);

-- junction table --> which product is in which warehouse and how many

CREATE TABLE inventory (

   id              SERIAL PRIMARY KEY,

   product_id      INTEGER NOT NULL REFERENCES products(id) ON DELETE RESTRICT,

   warehouse_id    INTEGER NOT NULL REFERENCES warehouses(id) ON DELETE RESTRICT,

   quantity        INTEGER NOT NULL DEFAULT 0 CHECK (quantity >= 0),

   created_at      TIMESTAMP NOT NULL DEFAULT NOW(),

   updated_at      TIMESTAMP NOT NULL DEFAULT NOW(),

   UNIQUE (product_id, warehouse_id)

);

-- full audit trail of every inventory change

CREATE TABLE inventory_history (

   id              SERIAL PRIMARY KEY,

   product_id      INTEGER NOT NULL REFERENCES products(id),

   warehouse_id    INTEGER NOT NULL REFERENCES warehouses(id),

   change          INTEGER NOT NULL,

   quantity_after  INTEGER NOT NULL,

   reason          VARCHAR(100) NOT NULL,

   reference_id    INTEGER,

   created_at      TIMESTAMP NOT NULL DEFAULT NOW()
```

```sql
);

-- suppliers

CREATE TABLE suppliers (

   id              SERIAL PRIMARY KEY,

   name            VARCHAR(255) NOT NULL,

   contact_email   VARCHAR(255),

   contact_phone   VARCHAR(50),

   lead_time_days  INTEGER,

   created_at      TIMESTAMP NOT NULL DEFAULT NOW(),

   updated_at      TIMESTAMP NOT NULL DEFAULT NOW()

);

-- many-to-many: a product can have multiple suppliers, a supplier many products

CREATE TABLE product_suppliers (

   product_id      INTEGER NOT NULL REFERENCES products(id) ON DELETE CASCADE,

   supplier_id     INTEGER NOT NULL REFERENCES suppliers(id) ON DELETE CASCADE,

   is_preferred    BOOLEAN NOT NULL DEFAULT FALSE,

   supplier_sku    VARCHAR(100),

   unit_cost       NUMERIC(12, 4),

   PRIMARY KEY (product_id, supplier_id)

);

CREATE TABLE bundle_items (

   bundle_id       INTEGER NOT NULL REFERENCES products(id) ON DELETE CASCADE,

   component_id    INTEGER NOT NULL REFERENCES products(id) ON DELETE RESTRICT,

   quantity        INTEGER NOT NULL DEFAULT 1 CHECK (quantity > 0),

   PRIMARY KEY (bundle_id, component_id),

   CHECK (bundle_id != component_id)
```

```
);
```

2.2 Indexes

```
-- company → warehouse lookups
CREATE INDEX idx_warehouses_company_id ON warehouses(company_id);

-- inventory joins
CREATE INDEX idx_inventory_product_id ON inventory(product_id);
CREATE INDEX idx_inventory_warehouse_id ON inventory(warehouse_id);

-- low-stock alert filtering-> "find all inventory below threshold"
CREATE INDEX idx_inventory_low_stock ON inventory(warehouse_id, quantity);

-- inventory history: time-range queries per product
CREATE INDEX idx_inventory_history_product_time ON inventory_history(product_id, created_at DESC);

-- sales activity detection: recent changes by warehouse
CREATE INDEX idx_inventory_history_warehouse_time ON inventory_history(warehouse_id, created_at DESC);

-- preferred supplier lookups
CREATE INDEX idx_product_suppliers_product ON product_suppliers(product_id) WHERE is_preferred =
TRUE;
```

2.3 Design Decisions

Why NUMERIC(12,4) for price?

Float types (FLOAT, REAL) have binary rounding errors. NUMERIC is exact. Four decimal places handles micro-pricing and currency conversions. This is non-negotiable for financial data.

Why quantity_after in inventory_history?

Storing the snapshot avoids replaying the full event log to reconstruct stock at a point in time. This makes analytics and debugging O(1) instead of O(n).

Why reason and reference_id in history?

A change of -5 is meaningless without context. Was it a sale, a write-off, a manual adjustment? reference_id links to the source order or PO, enabling full traceability essential for audits.

Why is_preferred on product_suppliers?

A product can have multiple suppliers, but the low-stock alert needs to surface one actionable contact. The preferred flag lets the business configure this without hardcoding logic.

Why ON DELETE RESTRICT on inventory?

You should never be able to delete a product that still has stock. RESTRICT enforces this at the DB level a last line of defense even if the application layer has a bug.

Why low_stock_threshold on the products table?

The business rule says thresholds vary by product type. Storing it on the product gives per-product configurability without a separate lookup table. It can be overridden per product regardless of type.

2.4 Gaps (Questions I'd Ask the Product Team)

1. **How is "recent sales activity" defined?** Last 7 days? 30 days? Is this configurable per company or global?
2. **Is stock tracked per-company or per-warehouse?** If a company moves stock between its own warehouses, is that a "sale"?
3. **How do bundles affect stock?** When a bundle is sold, does it decrement each component's inventory automatically, or is that handled manually?
4. **Can a warehouse belong to multiple companies?** (e.g., shared 3PL warehouse) affects the schema significantly.
5. **What happens when a product is discontinued?** Soft delete (is_active = false) or hard delete? What about historical orders?
6. **Are there user roles?** (warehouse staff vs. company admin vs. super admin) affects row-level security design.
7. **Is multi-currency needed?** If companies operate internationally, price needs a currency_code column.

8. **Who can update thresholds?** Product managers? Warehouse managers? This affects the access model.
9. **What's the expected data volume?** Millions of inventory_history rows change indexing and archival strategy.
10. **Are there SLA requirements for the alert endpoint?** This determines whether we need caching or a materialized view.

Part 3: API Implementation Low Stock Alerts

3.1 Assumptions

- "Recent sales activity" = at least one outbound inventory change (negative change in inventory_history) in the last **30 days**. This is configurable via query param.
- The alert fires per warehouse, not per product globally, a product at 2 units in Warehouse A is an alert even if it has 500 units in Warehouse B.
- days_until_stockout is calculated from average daily consumption over the recent activity window. If no recent sales, it returns null.
- Only the **preferred supplier** is returned. If no preferred supplier exists, the first supplier is used.
- Bundles are excluded from alerts (their components are tracked individually).
- Company must exist, otherwise 404.

3.2 Implementation

```
from flask import jsonify, request

from sqlalchemy import text

from datetime import datetime, timedelta

import logging

logger = logging.getLogger(__name__)

@app.route('/api/companies/<int:company_id>/alerts/low-stock', methods=['GET'])

def get_low_stock_alerts(company_id):

    """

    Returns low-stock alerts for all warehouses belonging to a company.

    Alerts fire when current stock <= product's low_stock_threshold AND

    the product has had sales activity within the lookback window.
```

```
    """

    # --- validate company exists ---

    company = Company.query.get(company_id)

    if not company:

        return jsonify({"error": f"Company {company_id} not found"}), 404

    # --- configurable lookback window (default 30 days) ---

    try:

        lookback_days = int(request.args.get('lookback_days', 30))

        if not (1 <= lookback_days <= 365):

            raise ValueError()

    except (ValueError, TypeError):

        return jsonify({"error": "lookback_days must be an integer between 1 and
365"}), 400

    activity_cutoff = datetime.utcnow() - timedelta(days=lookback_days)

    try:

        alerts = _build_alerts(company_id, activity_cutoff, lookback_days)

        return jsonify({

            "alerts": alerts,

            "total_alerts": len(alerts),

            "generated_at": datetime.utcnow().isoformat() + "Z",

            "lookback_days": lookback_days

        }), 200



    except Exception as e:

        logger.error(f"Error fetching low-stock alerts for company {company_id}:
{e}", exc_info=True)

        return jsonify({"error": "Internal server error"}), 500
```

```python
def _build_alerts(company_id: int, activity_cutoff: datetime, lookback_days: int)
-> list:
    """
    Core alert query. Uses a single SQL query with subqueries to:
    1. Filter to this company's warehouses
    2. Find inventory below threshold
    3. Confirm recent sales activity
    4. Calculate daily consumption rate for days_until_stockout
    5. Join preferred supplier info
    Returns a list of alert dicts.
    """

    # raw SQL for performance — ORM joins across 5 tables get slow
    # this query is read-only and parameterized (no SQL injection risk)
    query = text("""
        WITH

        -- Step 1: All warehouses for this company
        company_warehouses AS (
            SELECT id, name
            FROM warehouses
            WHERE company_id = :company_id
              AND is_active = TRUE
        ),
```

```sql
-- Step 2: Inventory records below threshold, excluding bundles
low_stock AS (
    SELECT
        i.id          AS inventory_id,
        i.product_id,
        i.warehouse_id,
        i.quantity    AS current_stock,
        p.name        AS product_name,
        p.sku,
        p.low_stock_threshold AS threshold
    FROM inventory i
    JOIN products p ON p.id = i.product_id
    JOIN company_warehouses cw ON cw.id = i.warehouse_id
    WHERE i.quantity <= p.low_stock_threshold
      AND p.is_active = TRUE
      AND p.is_bundle = FALSE   -- bundles tracked via components
),

-- Step 3: Recent outbound activity per product+warehouse
recent_activity AS (
    SELECT
        product_id,
        warehouse_id,
        COUNT(*)                        AS sale_count,
        SUM(ABS(change))                AS total_units_sold,
        MIN(created_at)                 AS first_sale,
```

```
            MAX(created_at)                 AS last_sale

        FROM inventory_history

        WHERE created_at >= :activity_cutoff

          AND change < 0              -- outbound only (sales, not restocks)

          AND reason IN ('sale', 'adjustment')

        GROUP BY product_id, warehouse_id

    ),



    -- Step 4: Preferred supplier per product (fallback to first if none
preferred)

    ranked_suppliers AS (

        SELECT

            ps.product_id,

            s.id          AS supplier_id,

            s.name        AS supplier_name,

            s.contact_email,

            ps.is_preferred,

            ROW_NUMBER() OVER (

                PARTITION BY ps.product_id

                ORDER BY ps.is_preferred DESC, ps.supplier_id ASC

            ) AS rn

        FROM product_suppliers ps

        JOIN suppliers s ON s.id = ps.supplier_id

    )



    -- Final join: only return products with recent activity
```

```
    SELECT
        ls.product_id,
        ls.product_name,
        ls.sku,
        ls.warehouse_id,
        cw.name             AS warehouse_name,
        ls.current_stock,
        ls.threshold,
        ra.total_units_sold,
        ra.sale_count,
        :lookback_days      AS days_in_window,
        rs.supplier_id,
        rs.supplier_name,
        rs.contact_email
    FROM low_stock ls
    JOIN company_warehouses cw ON cw.id = ls.warehouse_id
    JOIN recent_activity ra
        ON  ra.product_id   = ls.product_id
        AND ra.warehouse_id = ls.warehouse_id
    LEFT JOIN ranked_suppliers rs
        ON  rs.product_id   = ls.product_id
        AND rs.rn           = 1
    ORDER BY ls.current_stock ASC   -- most critical first
""")


rows = db.session.execute(query, {
```

```
    "company_id":      company_id,
    "activity_cutoff": activity_cutoff,
    "lookback_days":   lookback_days
}).fetchall()


alerts = []
for row in rows:
    # calculate days until stockout from average daily consumption
    days_until_stockout = _calc_days_until_stockout(
        current_stock=row.current_stock,
        total_units_sold=row.total_units_sold,
        days_in_window=lookback_days
    )


    # build supplier block — null-safe (product may have no supplier)
    supplier = None
    if row.supplier_id:
        supplier = {
            "id":            row.supplier_id,
            "name":          row.supplier_name,
            "contact_email": row.contact_email
        }


    alerts.append({
        "product_id":          row.product_id,
        "product_name":        row.product_name,
```

```
            "sku":                  row.sku,
            "warehouse_id":         row.warehouse_id,
            "warehouse_name":       row.warehouse_name,
            "current_stock":        row.current_stock,
            "threshold":            row.threshold,
            "days_until_stockout": days_until_stockout,
            "supplier":             supplier
        })

    return alerts


def _calc_days_until_stockout(current_stock: int, total_units_sold: int,
days_in_window: int):
    """
    Estimate days until stockout based on average daily consumption.
    Returns None if no consumption data (avoid division by zero).
    Returns 0 if already out of stock.
    """
    if current_stock <= 0:
        return 0
    if total_units_sold == 0 or days_in_window == 0:
        return None  # no sales data — can't predict

    avg_daily_consumption = total_units_sold / days_in_window
```

```python
# protect against float edge cases
if avg_daily_consumption <= 0:
    return None

days = current_stock / avg_daily_consumption
return round(days, 1)
```

3.3 Edge Cases Handled

Edge Case	How It's Handled
Company doesn't exist	Returns 404 before any DB query
Invalid lookback_days param	Validated to integer, 1–365 range, returns 400
Product below threshold but no recent sales	Excluded by JOIN recent_activity (only appears if recent outbound activity exists)
Product has no supplier	LEFT JOIN on suppliers; supplier field returns null in response
Multiple suppliers	ROW_NUMBER() picks preferred supplier, falls back to first
Product at zero stock	days_until_stockout returns 0, not a negative number
No sales data for consumption rate	days_until_stockout returns null instead of dividing by zero
Bundle products	Excluded via is_bundle = FALSE — their components are tracked separately

Inactive warehouses	Filtered via is_active = TRUE on warehouses
Inactive products	Filtered via is_active = TRUE on products
DB error	Caught by outer except, logged, returns clean 500
Most critical stock first	ORDER BY current_stock ASC — worst case surfaces at top

---

3.4 What I'd Add Before Shipping to Production

Pagination

A company with 50 warehouses and 10,000 products could return thousands of alerts. Add ?page=1&page_size=50.

```
# Add to query: LIMIT :limit OFFSET :offset

page = int(request.args.get('page', 1))

page_size = min(int(request.args.get('page_size',
50)), 200)  # cap at 200

offset = (page - 1) * page_size
```

Caching

This endpoint may be hit frequently by dashboards. A 60-second Redis cache keyed on company_id + lookback_days would dramatically reduce DB load.

Filtering

Allow ?warehouse_id=456 to scope alerts to one warehouse, and ?severity=critical (e.g., stock = 0) for triage views.

Async / Background Job

For large companies, computing this on every request is wasteful. A scheduled job (e.g., Celery every 15 minutes) could pre-compute alerts and store them, making the endpoint a simple read.

Summary of Key Decisions

Decision	Rationale
Single transaction for product creation	Prevents partial writes the most dangerous class of bug in inventory systems
NUMERIC(12,4) for price	Float rounding errors are unacceptable in financial data
quantity_after snapshot in history	O(1) point-in-time reconstruction instead of replaying event log
Per-product low_stock_threshold	Business rule says thresholds vary store it where it's queried
Raw SQL for alert query	5-table join with CTEs performs significantly better than chained ORM calls
is_preferred on product_suppliers	One actionable contact per alert without hardcoded priority logic
Alerts ordered by current_stock ASC	Most critical items surface first aligns with how warehouse managers triage